Student course view — implementation guide

Companion document to `student_course_view_branded.html`. This guide is written for the existing `api/` + `web/` workspaces of `India-Learns-LMS` and the brand tokens already in use on the faculty side.

Stack assumed (from repo README): Node 20 LTS, TypeScript 5.4 (ESM), Express 4, Mongoose 8, MongoDB 7, React 18, Vite 5, Tailwind 3, React Router 6.

---

1. What's changing

The current student course page renders one flat list of all assignments under the course, plus a separate flat list of session titles with no progress information. The redesign restructures the page around four ideas:

1. **The course is a Module → Session → Assignment hierarchy.** Modules are the spine of the page. Sessions live inside their parent module. Assignments live inside their parent session. The flat assignment list is removed entirely.
2. **Progress is visible at every level.** Course-level (top strip), module-level (each module's chapter header carries a fraction), session-level (per-session progress bar with one segment per assignment), and assignment-level (status icon and pill on every row).
3. **Triage is a first-class job.** Three status cards (Late / Due soon / Upcoming) and a "Needs your attention" panel pull the few assignments that need action to the top of the page, so students don't scroll through 21 rows to find the overdue one.
4. **The student's location is unmissable.** A horizontal "journey" stepper across the top shows all 4 modules + the capstone exam as a path, with the current module marked "You are here." That signal repeats in the progress strip ("Currently in Module 1 · Foundations") and again on the module's chapter header ("Module 1 · In progress" with an orange accent rail in the left margin).

The course description, which currently dominates the viewport, collapses to a 2-sentence preview with a "Read more" toggle. Each session card always shows its assignment preview inline (no expand/collapse) — the session header is itself a navigation link to the session detail page.

---

2. Data model

The faculty side already operates on a Module → Session hierarchy, so the `Module` model very

likely already exists in `api/src/models/`. Confirm before adding anything new. The shape below is what the student endpoint relies on.

2.1 Module schema

ts

```ts
// api/src/models/Module.ts
const ModuleSchema = new Schema({
  courseId: { type: Schema.Types.ObjectId, ref: 'Course', required: true, index: tru
  order:    { type: Number, required: true },                // 1, 2, 3, 4 — drives
  title:    { type: String, required: true },                // e.g. "Mathematical f
  subtitle: { type: String, default: '' },                   // optional short blurb
}, { timestamps: true });

ModuleSchema.index({ courseId: 1, order: 1 });
```

2.2 Session schema

Sessions belong to a module. Add `moduleId` if it isn't already there.

ts

```ts
// api/src/models/Session.ts
const SessionSchema = new Schema({
  courseId: { type: Schema.Types.ObjectId, ref: 'Course',  required: true, index: tr
  moduleId: { type: Schema.Types.ObjectId, ref: 'Module',  required: true, index: tr
  order:    { type: Number, required: true },               // order within the modu
  title:    { type: String, required: true },
  subtitle: { type: String, default: '' },                   // short topic blurb sho
}, { timestamps: true });

SessionSchema.index({ courseId: 1, moduleId: 1, order: 1 });
```

2.3 Assignment schema

Assignments belong to a session. Add `sessionId` if it isn't already there.

ts

```
// api/src/models/Assignment.ts
const AssignmentSchema = new Schema({
  courseId:    { type: Schema.Types.ObjectId, ref: 'Course',  required: true, index:
  sessionId:   { type: Schema.Types.ObjectId, ref: 'Session', required: true, index:
  title:       { type: String, required: true },
  description: { type: String, required: true },
  dueAt:       { type: Date,   required: true },
  maxPoints:   { type: Number, required: true, default: 100 },
}, { timestamps: true });

AssignmentSchema.index({ courseId: 1, sessionId: 1, dueAt: 1 });
```

2.4 Submission states

Status derivation depends on submission state. The state enum should be:

```
draft | submitted | graded | returned
```

If submission state is currently a boolean, replace with an enum.

2.5 Migration

If `Assignment.sessionId` or `Session.moduleId` doesn't exist yet, add a script under `scripts/migrations/`:

ts

```
// scripts/migrations/001-link-hierarchy.ts
// npx tsx scripts/migrations/001-link-hierarchy.ts
import { connect } from 'mongoose';
import { Assignment } from '../../api/src/models/Assignment.js';
import { Session } from '../../api/src/models/Session.js';

async function run() {
  await connect(process.env.MONGODB_URI!);

  const orphanedAssignments = await Assignment.find({ sessionId: { $exists: false }
  console.log(`Linking ${orphanedAssignments.length} assignments to sessions…`);
  // Build the mapping (likely manual or via the curriculum-import scripts).
  // Long-term: update curriculum-import/ to write moduleId/sessionId on creation.
}
run().catch(console.error);
```

For new courses, the `curriculum-import/` scripts should write `moduleId` and `sessionId` at creation time so this is never needed again.

---

3. API contract

The page makes one call:

```
GET /api/student/courses/:courseId
```

The response is fully nested — modules contain sessions, sessions contain assignments, plus pre-computed rollups so the client does no derivation.

3.1 Shared types

```ts
```

```ts
// packages/shared-types/src/student.ts
export type AssignmentStatus =
  | 'graded'      // submitted and graded — show score pill
  | 'submitted'   // submitted, awaiting grading
  | 'late'        // past dueAt, no submission
  | 'dueSoon'     // within next 7 days, no submission
  | 'upcoming';   // > 7 days out, no submission

export interface AssignmentSummaryDTO {
  id: string;
  title: string;
  dueAt: string;            // ISO
  maxPoints: number;
  score: number | null;     // null until graded
  status: AssignmentStatus;
  daysUntilDue: number;     // negative if late
}

export type SessionState = 'not_started' | 'in_progress' | 'complete';

export interface SessionSummaryDTO {
  id: string;
  order: number;
  title: string;
  subtitle: string;
  state: SessionState;
  assignments: AssignmentSummaryDTO[];
  progress: {
    total: number;
    completed: number;       // status === 'graded'
    late: number;
    dueSoon: number;
  };
}

export type ModuleState = SessionState;

export interface ModuleSummaryDTO {
  id: string;
  order: number;
  title: string;
  subtitle: string;
  state: ModuleState;
```

```ts
  sessions: SessionSummaryDTO[];
  progress: {
    total: number;           // total assignments across all sessions in module
    completed: number;
  };
}

export interface StudentCourseViewDTO {
  course: {
    id: string;
    title: string;
    type: 'sandbox' | 'live';   // drives the "SANDBOX" eyebrow label
    description: string;
  };
  progress: {
    totalAssignments: number;
    completedAssignments: number;
    percentComplete: number;     // 0-100
    currentModuleOrder: number;
    currentModuleTitle: string;  // for the "Currently in Module 1 · Foundations" li
    finalExamDueAt: string | null;
  };
  counts: {
    late: number;
    dueSoon: number;
    upcoming: number;
  };
  needsAttention: AssignmentSummaryDTO[];   // pre-sorted: late first, then nearest
  modules: ModuleSummaryDTO[];              // sorted by order
}
```

3.2 Status derivation

Compute status server-side so the client doesn't need to know the rules.

```ts
```

```ts
// api/src/services/studentCourseView.ts
export function deriveAssignmentStatus(
  a: AssignmentDoc,
  sub: SubmissionDoc | null,
  now: Date,
): AssignmentStatus {
  if (sub?.state === 'graded')    return 'graded';
  if (sub?.state === 'submitted') return 'submitted';

  const msPerDay = 1000 * 60 * 60 * 24;
  const daysUntilDue = Math.floor((a.dueAt.getTime() - now.getTime()) / msPerDay);

  if (daysUntilDue < 0)  return 'late';
  if (daysUntilDue <= 7) return 'dueSoon';
  return 'upcoming';
}

export function deriveState(progress: { total: number; completed: number; late: numb
  if (progress.total > 0 && progress.completed === progress.total) return 'complete'
  if (progress.completed > 0 || progress.late > 0 || progress.dueSoon > 0) return 'i
  return 'not_started';
}
```

3.3 Express handler

ts

```ts
// api/src/routes/student/courseView.ts
router.get('/courses/:courseId', requireStudent, async (req, res) => {
  const { courseId } = req.params;
  const studentId = req.user.id;
  const now = new Date();

  const [course, modules, sessions, assignments, submissions] = await Promise.all([
    Course.findById(courseId).lean(),
    Module.find({ courseId }).sort({ order: 1 }).lean(),
    Session.find({ courseId }).sort({ order: 1 }).lean(),
    Assignment.find({ courseId }).sort({ dueAt: 1 }).lean(),
    Submission.find({ studentId, courseId }).lean(),
  ]);

  if (!course) return res.status(404).json({ error: 'Course not found' });

  const submissionByAssignment = new Map(submissions.map(s => [s.assignmentId.toStri

  // Build assignment DTOs
  const assignmentDTOs = assignments.map(a => buildAssignmentDTO(a, submissionByAssi
  const assignmentByIdSession = new Map<string, AssignmentSummaryDTO[]>();
  assignmentDTOs.forEach(dto => {
    const sid = (dto as any).sessionId; // attach during build
    if (!assignmentByIdSession.has(sid)) assignmentByIdSession.set(sid, []);
    assignmentByIdSession.get(sid)!.push(dto);
  });

  // Build session DTOs grouped by module
  const sessionsByModule = new Map<string, SessionSummaryDTO[]>();
  sessions.forEach(s => {
    const sessionAssignments = assignmentByIdSession.get(s._id.toString()) ?? [];
    const progress = {
      total: sessionAssignments.length,
      completed: sessionAssignments.filter(a => a.status === 'graded').length,
      late: sessionAssignments.filter(a => a.status === 'late').length,
      dueSoon: sessionAssignments.filter(a => a.status === 'dueSoon').length,
    };
    const sessionDTO: SessionSummaryDTO = {
      id: s._id.toString(),
      order: s.order,
      title: s.title,
      subtitle: s.subtitle ?? '',
      state: deriveState(progress),
```

```
    assignments: sessionAssignments,
    progress,
  };
  const mid = s.moduleId.toString();
  if (!sessionsByModule.has(mid)) sessionsByModule.set(mid, []);
  sessionsByModule.get(mid)!.push(sessionDTO);
});

// Build module DTOs
const moduleDTOs: ModuleSummaryDTO[] = modules.map(m => {
  const moduleSessions = sessionsByModule.get(m._id.toString()) ?? [];
  const total     = moduleSessions.reduce((sum, s) => sum + s.progress.total, 0);
  const completed = moduleSessions.reduce((sum, s) => sum + s.progress.completed,
  const late      = moduleSessions.reduce((sum, s) => sum + s.progress.late, 0);
  const dueSoon   = moduleSessions.reduce((sum, s) => sum + s.progress.dueSoon, 0)
  return {
    id: m._id.toString(),
    order: m.order,
    title: m.title,
    subtitle: m.subtitle ?? '',
    state: deriveState({ total, completed, late, dueSoon }),
    sessions: moduleSessions,
    progress: { total, completed },
  };
});

// Course-level rollups
const counts = {
  late:     assignmentDTOs.filter(a => a.status === 'late').length,
  dueSoon:  assignmentDTOs.filter(a => a.status === 'dueSoon').length,
  upcoming: assignmentDTOs.filter(a => a.status === 'upcoming').length,
};

const needsAttention = assignmentDTOs
  .filter(a => a.status === 'late' || a.status === 'dueSoon')
  .sort((a, b) => a.daysUntilDue - b.daysUntilDue)   // late items have negative d
  .slice(0, 5);

const currentModule = moduleDTOs.find(m => m.state === 'in_progress')
                    ?? moduleDTOs.find(m => m.state === 'not_started')
                    ?? moduleDTOs[0];

res.json({
  course: { id: course._id, title: course.title, type: course.type, description: c
```

```
    progress: {
      totalAssignments: assignmentDTOs.length,
      completedAssignments: assignmentDTOs.filter(a => a.status === 'graded').length
      percentComplete: assignmentDTOs.length === 0 ? 0
        : Math.round((assignmentDTOs.filter(a => a.status === 'graded').length / ass
      currentModuleOrder: currentModule?.order ?? 1,
      currentModuleTitle: currentModule?.title ?? '',
      finalExamDueAt: course.finalExamDueAt ?? null,
    },
    counts,
    needsAttention,
    modules: moduleDTOs,
  });
});
```

---

4. Frontend

4.1 Tailwind tokens (matches faculty)

Both faculty and student pages share one design system. Extend `web/tailwind.config.js` once with the same tokens the faculty handoff doc already uses:

```
js
```

```
module.exports = {
  theme: {
    extend: {
      colors: {
        navy:   { DEFAULT: '#132B5E', dark: '#0B1C42' },
        orange: { DEFAULT: '#F28C2C', dark: '#D96F10' },
        cream:  { DEFAULT: '#FAF6EE', dark: '#F1EADA' },
        ink:    '#1A2540',
        muted:  '#6B7280',
        success: { DEFAULT: '#2F6D3C', bg: '#E8F1E9' },
        warning: { DEFAULT: '#A35500', bg: '#FDF0DE' },
        danger:  { DEFAULT: '#B91C1C', bg: '#FEE2E2' },
      },
      fontFamily: {
        display: ['Fraunces', 'Georgia', 'serif'],
        sans:    ['Geist', 'system-ui', 'sans-serif'],
      },
      borderRadius: {
        // base Tailwind already provides md (8) and xl (12); cards use xl, buttons
      },
    },
  },
};
```

Add the Google Fonts links to `web/index.html` (or load via `vite-plugin-fonts` for production):

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Fraunces:ital,opsz,wght,SOFT@0,
```

Apply Fraunces with `font-variation-settings: "opsz" 144, "SOFT" 30` on the largest titles, smaller `opsz` values for smaller display text — see the mockup CSS for the exact pairings used at each size.

4.2 Component tree

Create under `web/src/pages/student/course-view/`:

```
StudentCoursePage.tsx                ← top-level page, fetches the DTO
├── CourseHeader.tsx                 ← eyebrow, title, collapsible description
├── ProgressStrip.tsx                ← percent, totals, "Currently in Module X"
line, exam pill
├── ModuleJourney.tsx                ← horizontal stepper: 4 modules + capstone,
"You are here"
├── StatusCardsRow.tsx               ← Late / Due soon / Upcoming
├── NeedsAttentionPanel.tsx          ← top 3-5 action items
└── ModuleList.tsx
    └── ModuleSection.tsx            ← module header (eyebrow + name + fraction)
        └── SessionCard.tsx          ← static, navigation link, with assignment
preview
            └── AssignmentRow.tsx    ← icon + title + meta + score/due pill
```

There is no expand/collapse state anywhere in the session/module hierarchy. The entire course tree is rendered visible at all times. The only stateful interaction is the description "Read more" toggle on `CourseHeader`.

4.3 Routing

ts

```ts
<Route path="/courses/:courseId"                          element={<StudentCoursePage
<Route path="/courses/:courseId/sessions/:sessionId"      element={<StudentSessionPage
<Route path="/courses/:courseId/assignments/:assnId"      element={<StudentAssignmentP
```

The session card header is an `<a>` to the session route. Each assignment row is an `<a>` to the assignment route. Don't nest — the session header and the assignment rows are siblings inside the card div, not parent/child.

4.4 Reference components

tsx

```tsx
// web/src/pages/student/course-view/CourseHeader.tsx
import { useState } from 'react';

export function CourseHeader({ title, type, description }: {
  title: string; type: 'sandbox' | 'live'; description: string;
}) {
  const [expanded, setExpanded] = useState(false);
  const sentences = description.match(/[^.!?]+[.!?]+/g) ?? [description];
  const preview = sentences.slice(0, 2).join(' ').trim();
  const hasMore = sentences.length > 2;

  return (
    <section className="mb-9">
      <p className="font-display italic text-orange-dark text-[13px] tracking-wider
         style={{ fontVariationSettings: '"opsz" 14, "SOFT" 100' }}>
        {type}
      </p>
      <h1 className="font-display text-ink leading-[1.05] tracking-tight mb-5"
          style={{ fontSize: 'clamp(40px, 5.5vw, 60px)', fontVariationSettings: '"op
        {title}
      </h1>
      <p className="text-ink/80 text-base leading-[1.65] max-w-[68ch] mb-1.5">
        {expanded ? description : preview}
      </p>
      {hasMore && (
        <button onClick={() => setExpanded(e => !e)}
                className="text-[13px] text-navy hover:text-orange-dark font-medium
          {expanded ? 'Show less' : 'Read more'}
          <span className={`text-[11px] transition-transform ${expanded ? 'rotate-18
        </button>
      )}
    </section>
  );
}
```

tsx

```tsx
// web/src/pages/student/course-view/ModuleJourney.tsx
import type { ModuleSummaryDTO } from '@india-learns/shared-types';

interface Props {
  modules: ModuleSummaryDTO[];
  percentComplete: number;
  finalExamDueAt: string | null;
}

export function ModuleJourney({ modules, percentComplete, finalExamDueAt }: Props) {
  const totalCols = modules.length + (finalExamDueAt ? 1 : 0);
  return (
    <div className="mt-10 px-8 py-7 bg-white border border-cream-dark rounded-2xl ov
      <p className="font-display italic text-muted text-[13px] mb-6">— Your journey
      <div className="relative min-w-[520px]">
        <div className="absolute top-[22px] left-[22px] right-[22px] h-0.5 bg-cream-
        <div className="absolute top-[22px] left-[22px] h-0.5 bg-orange rounded-full
             style={{ width: `${percentComplete}%` }} />
        <div className="grid relative z-10" style={{ gridTemplateColumns: `repeat(${
          {modules.map(m => <ModuleNode key={m.id} module={m} />)}
          {finalExamDueAt && <ExamNode />}
        </div>
      </div>
    </div>
  );
}

function ModuleNode({ module: m }: { module: ModuleSummaryDTO }) {
  const isCurrent = m.state === 'in_progress';
  return (
    <button onClick={() => scrollToModule(m.id)}
            className="flex flex-col items-center gap-3 hover:-translate-y-0.5 trans
      <div className={`w-11 h-11 rounded-full grid place-items-center font-display f
                       ${isCurrent
                         ? 'bg-white border-2 border-orange text-orange-dark font-se
                         : 'bg-white border-[1.5px] border-cream-dark text-muted'}`}
           style={{ fontVariationSettings: '"opsz" 14, "SOFT" 30' }}>
        {m.order}
      </div>
      <span className={`text-[10px] tracking-[0.1em] uppercase font-semibold -mb-1
                        ${isCurrent ? 'text-orange-dark' : 'text-muted'}`}>
        {isCurrent ? 'You are here' : 'Module'}
      </span>
```

```tsx
      <span className={`text-xs text-center leading-tight max-w-[100px] font-medium
                        ${isCurrent ? 'text-ink font-semibold' : 'text-muted'}`}>
        {m.title}
      </span>
      {isCurrent && (
        <span className="text-[11px] text-orange-dark font-semibold tabular-nums">
          {m.progress.completed}/{m.progress.total}
        </span>
      )}
    </button>
  );
}

function ExamNode() { /* navy-filled dot with cap icon, "Final" eyebrow, "Capstone e

function scrollToModule(id: string) {
  document.getElementById(`module-${id}`)?.scrollIntoView({ behavior: 'smooth', bloc
}
```

```tsx
```

```tsx
// web/src/pages/student/course-view/ModuleSection.tsx
import type { ModuleSummaryDTO } from '@india-learns/shared-types';
import { SessionCard } from './SessionCard';

export function ModuleSection({ module: m }: { module: ModuleSummaryDTO }) {
  const isCurrent = m.state === 'in_progress';
  const stateLabel =
    m.state === 'complete' ? 'Complete' :
    m.state === 'in_progress' ? 'In progress' :
    'Not started';

  return (
    <section id={`module-${m.id}`} className={`relative ${isCurrent ? 'before:conten
      <header className={`grid grid-cols-[1fr_auto] gap-4 items-end pt-[18px] pb-4 m
        <div className="min-w-0">
          <div className={`flex items-center gap-2 font-display italic text-[11px] t
                          ${isCurrent ? 'text-orange-dark' : 'text-muted'}`}
               style={{ fontVariationSettings: '"opsz" 14, "SOFT" 100' }}>
            <span className={`w-[7px] h-[7px] rounded-full ${isCurrent ? 'bg-orange
            Module {m.order} · {stateLabel}
          </div>
          <h3 className="font-display text-[22px] leading-[1.2] tracking-tight text-
              style={{ fontVariationSettings: '"opsz" 36, "SOFT" 30, "wght" 460' }}>
            {m.title}
          </h3>
        </div>
        <div className="text-right pb-0.5">
          <div className={`text-sm font-semibold tabular-nums ${isCurrent ? 'text-or
            {m.progress.completed} / {m.progress.total}
          </div>
          <div className="text-[11px] text-muted mt-0.5 tracking-wider uppercase fon
        </div>
      </header>

      {m.sessions.map(s => <SessionCard key={s.id} session={s} />)}
    </section>
  );
}
```

tsx

```tsx
// web/src/pages/student/course-view/SessionCard.tsx
import { Link } from 'react-router-dom';
import type { SessionSummaryDTO } from '@india-learns/shared-types';
import { AssignmentRow } from './AssignmentRow';

export function SessionCard({ session: s }: { session: SessionSummaryDTO }) {
  const isCurrent = s.state === 'in_progress';

  return (
    <article className={`bg-white rounded-xl mb-3 overflow-hidden transition-colors
                         ${isCurrent ? 'border-[1.5px] border-orange' : 'border bord
      <Link to={`./sessions/${s.id}`}
            className={`flex items-center gap-[18px] px-[22px] py-[18px] no-underlin
                        ${isCurrent ? 'hover:bg-orange/[0.05]' : 'hover:bg-cream'} g
        <SessionNumber order={s.order} state={s.state} />
        <div className="flex-1 min-w-0">
          <div className="font-display text-[17px] leading-[1.25] tracking-tight tex
               style={{ fontVariationSettings: '"opsz" 24, "SOFT" 30, "wght" 480' }}
            {s.title}
          </div>
          <div className={`text-xs flex items-center gap-2 ${isCurrent ? 'text-orang
            <span className={`w-1.5 h-1.5 rounded-full ${isCurrent ? 'bg-orange-dark
            {sessionStateLabel(s.state)} · {s.subtitle}
          </div>
        </div>
        <div className="flex flex-col items-end gap-2 min-w-[130px]">
          <span className={`text-[13px] font-semibold tabular-nums ${isCurrent ? 'te
            {s.progress.completed} / {s.progress.total}
          </span>
          <ProgressSegments assignments={s.assignments} />
        </div>
        <ChevronRight className="text-muted transition-all group-hover:text-navy gro
      </Link>

      {/* Always-visible assignment preview */}
      <div className="border-t border-cream-dark bg-cream">
        <div className="px-[22px] pt-2.5 pb-1.5 pl-20 text-[10px] tracking-[0.08em]
          Assignments · {s.assignments.length}
        </div>
        {s.assignments.map(a => <AssignmentRow key={a.id} assignment={a} />)}
      </div>
    </article>
```

```
  );
}
```

tsx

```tsx
// web/src/pages/student/course-view/AssignmentRow.tsx
import { Link } from 'react-router-dom';
import type { AssignmentSummaryDTO } from '@india-learns/shared-types';

export function AssignmentRow({ assignment: a }: { assignment: AssignmentSummaryDTO
  return (
    <Link to={`../assignments/${a.id}`}
          className="flex items-center gap-3.5 px-[22px] py-3 pl-20 border-t border-
      <StatusIcon status={a.status} />
      <div className="flex-1 min-w-0">
        <div className="text-sm font-medium text-ink truncate">{a.title}</div>
        <div className={`text-xs ${metaColor(a.status)}`}>{metaLine(a)}</div>
      </div>
      {a.status === 'graded' && a.score !== null
        ? <span className="text-xs font-semibold px-[11px] py-1 bg-success-bg text-s
            {a.score} / {a.maxPoints}
          </span>
        : <DuePill status={a.status} daysUntilDue={a.daysUntilDue} />
      }
    </Link>
  );
}

function metaLine(a: AssignmentSummaryDTO): string {
  if (a.status === 'graded')   return 'Graded · feedback available';
  if (a.status === 'submitted') return 'Submitted · awaiting grade';
  if (a.status === 'late')      return `Late · was due ${formatDate(a.dueAt)}`;
  return `Due ${formatDate(a.dueAt)}`;
}
```

The `ProgressSegments`, `SessionNumber`, `StatusIcon`, `DuePill`, and `formatDate` helpers are straightforward — see the SVG markup in the mockup HTML for exact icon/dot dimensions. Keep all of them under `course-view/components/`.

4.5 Data fetching

Match whatever pattern the rest of the app uses. If TanStack Query isn't yet standardised:

tsx

```tsx
// web/src/pages/student/course-view/StudentCoursePage.tsx
import { useQuery } from '@tanstack/react-query';
import { useParams } from 'react-router-dom';

export function StudentCoursePage() {
  const { courseId } = useParams<{ courseId: string }>();
  const { data, isLoading, error } = useQuery({
    queryKey: ['student-course-view', courseId],
    queryFn: () => fetch(`/api/student/courses/${courseId}`).then(r => r.json()),
    staleTime: 30_000,
  });

  if (isLoading) return <CoursePageSkeleton />;
  if (error || !data) return <CoursePageError />;

  return (
    <main className="max-w-[980px] mx-auto px-8 py-10 pb-20">
      <BackLink to="/courses" />
      <CourseHeader {...data.course} />
      <ProgressStrip progress={data.progress} />
      <ModuleJourney modules={data.modules}
                     percentComplete={data.progress.percentComplete}
                     finalExamDueAt={data.progress.finalExamDueAt} />
      <StatusCardsRow counts={data.counts} />
      <NeedsAttentionPanel assignments={data.needsAttention} />
      <SectionHead title="Course content" meta={`${data.modules.length} modules · ${
      {data.modules.map(m => <ModuleSection key={m.id} module={m} />)}
    </main>
  );
}
```

---

5. Mobile (PWA)

The mockup is responsive at the 720px breakpoint. Specifically:

- The journey strip is horizontally scrollable on phones via `overflow-x-auto` on the container plus `min-w-[520px]` on the inner track.
- Status cards collapse to a single column.
- Session card padding tightens, the session number badge shrinks from 40 → 36px.

- Assignment row left padding goes from 80 → 60px.
- The current module's left accent rail moves from `-left-4` to `-left-2` so it doesn't sit outside the page edge.
- The course title uses `clamp(30px, 8vw, 38px)` on mobile.
- The top nav drops the search input and the user's name/role text — only the avatar stays.

---

6. Recommended sequencing

Ship in 4 PRs rather than one giant change:

1. **Backend foundation.** Confirm `Module` model exists; add `Session.moduleId` and `Assignment.sessionId` if missing; write the migration; update the curriculum-import scripts. Nothing user-visible. (One PR.)
2. **API contract.** Implement the new `GET /api/student/courses/:courseId` returning the nested `StudentCourseViewDTO`. Update the existing student page to consume the new shape but keep the old layout — confirms the API works end-to-end. (One PR.)
3. **Visual rebuild.** Land the new components. Old page deleted in the same PR. (One PR.)
4. **Polish.** Skeleton loading state, empty states (course with 0 modules), error states, keyboard navigation through journey nodes. (One PR.)

Steps 1 and 2 are invisible to students but make step 3 a clean swap.

---

7. Open product questions

A few things worth resolving before the visual rebuild lands:

- **What counts as "late"?** The mockup uses `dueAt < now AND no submission`. Does a grace period apply? Does a draft submission count as "submitted"?
- **How are modules created and ordered?** If the curriculum-generator already produces modules (the faculty-side conversation referenced this), confirm its output writes `moduleId` consistently. Otherwise the courseroles will need a UI to define modules.
- **Locked content.** The mockup shows full assignment lists for all sessions, including future ones. If product doesn't want students seeing assignments before their session is "open," replace the upcoming-session preview with a count + first-assignment teaser, and put a lock icon in the session number badge.

- **Capstone modelling.** The journey treats the capstone exam as a separate node after the last module. Currently the data model has `course.finalExamDueAt`. If the capstone is actually just the last assignment of the last module, simplify and let the journey render N modules with the last one labelled "Capstone."
- **Journey node click target.** Currently scrolls to the module section in-page. Long-term, if module pages get materials/resources/discussion, consider making this a route to a dedicated module page instead.